

Điều hòa hành động trong thời gian tập luyện để phân chia thời gian thực hiệu quả

Kevin Black Allen Z. Ren Michael Equi Sergey Levine Trí tuệ thể chất

Tóm tắt—Phân đoạn thời gian thực (RTC) cho phép các mô hình hành động ngôn ngữ-tầm nhìn (VLA) tạo ra các quỹ đạo robot phản ứng mượt mà bằng cách dự đoán không đồng bộ các khối hành động và điều chỉnh các hành động đã cam kết trước đó thông qua tính năng inpainting trong thời gian suy luận. Tuy nhiên, phương pháp inpainting này đưa ra chi phí tính toán làm tăng độ trễ suy luận. Trong công việc này, chúng tôi đề xuất một giải pháp thay thế đơn giản: mô phỏng độ trễ suy luận tại thời điểm đào tạo và điều chỉnh trực tiếp các tiền tố hành động, loại bỏ mọi chi phí về thời gian suy luận. Phương pháp của chúng tôi không yêu cầu sửa đổi kiến trúc mô hình hoặc thời gian chạy của robot và có thể được triển khai chỉ với một vài dòng mã bổ sung. Trong các thử nghiệm mô phỏng, chúng tôi nhận thấy rằng RTC trong thời gian huấn luyện hoạt động tốt hơn RTC thời gian suy luận ở độ trễ suy luận cao hơn. Trong các thử nghiệm thực tế về nhiệm vụ đóng hộp và pha cà phê espresso với $\pi_{0.6}$ VLA, chúng tôi chứng minh rằng RTC trong thời gian đào tạo duy trì cả hiệu suất nhiệm vụ và tốc độ tương đương với RTC thời gian suy luận trong khi rẻ hơn về mặt tính toán. Kết quả của chúng tôi cho thấy rằng điều hòa hành động trong thời gian huấn luyện là một sự thay thế thực tế cho việc vẽ lại theo thời gian suy luận trong điều khiển robot thời gian thực.

I. GIỚI THIỆU

Không giống như chatbot hoặc công cụ tìm kiếm, các tác nhân được thể hiện phải hoạt động theo thời gian thực. Vòng phản hồi giữa hành động của tác nhân và môi trường của nó đòi hỏi khả năng phản ứng - giống như một vận động viên con người, tác nhân không thể đơn giản “dừng lại và suy nghĩ” trong khi thế giới bên ngoài thay đổi. Tuy nhiên, quy mô ngày càng tăng của các mô hình biên giới khiến việc này ngày càng trở nên khó khăn hơn. Không nơi nào điều này rõ ràng hơn trong lĩnh vực học tập của robot, nơi các mô hình hành động ngôn ngữ thị giác (VLA) bao gồm hàng tỷ tham số ngày càng được sử dụng để điều khiển robot ở tần số cao nhằm hoàn thành các nhiệm vụ khéo léo. Tạo ra các quỹ đạo mượt mà nhưng phản ứng nhanh khi độ trễ suy luận của mô hình ở mức hàng chục đến hàng trăm mili giây là một thách thức không hề nhỏ.

Phân đoạn thời gian thực (RTC; [5]) trình bày một cách tiếp cận cho vấn đề này kết hợp phân đoạn hành động [9, 27], khớp luồng [13] và inpainting theo thời gian suy luận [18, 21]. Trong RTC, các đoạn hành động được dự đoán không đồng bộ - đoạn tiếp theo được tạo trong khi đoạn hiện tại vẫn đang thực thi. Để đảm bảo tính liên tục giữa các khối, mỗi thế hệ được điều chỉnh dựa trên *tiền tố* cố định của các hành động được dự đoán trước đó, in sâu vào phần còn lại. Tuy nhiên, phương pháp inpainting theo thời gian suy luận được RTC sử dụng đưa ra thêm chi phí tính toán — và do đó có độ trễ — điều này phần nào phá vỡ mục đích của khung thực thi thời gian thực. Theo kinh nghiệm, chúng tôi cũng nhận thấy rằng việc inpainting trong thời gian suy luận về cơ bản bị hạn chế ở khả năng xử lý độ trễ suy luận cao.

Trong công việc này, chúng tôi tăng cường RTC bằng phương pháp inpainting mô phỏng độ trễ suy luận tại thời điểm đào tạo và loại bỏ mọi chi phí tính toán trong thời gian suy luận. Phương pháp của chúng tôi hoạt động

như một giải pháp thay thế ngẫu nhiên cho RTC thời gian suy luận: nó không yêu cầu sửa đổi kiến trúc mô hình hoặc thời gian chạy của robot và có thể được triển khai chỉ với một vài dòng mã bổ sung. Trên các điểm chuẩn mô phỏng, chúng tôi nhận thấy rằng RTC trong thời gian đào tạo hoạt động tốt hơn RTC trong thời gian suy luận ở độ trễ suy luận cao hơn. Trong thế giới thực, chúng tôi chứng minh rằng RTC trong thời gian đào tạo có thể được thêm thành công bằng cách tinh chỉnh mô hình cơ sở *không* được đào tạo trước với điều hòa tiền tố hành động. Bằng cách áp dụng RTC thời gian đào tạo cho $\pi_{0.6}$ VLA [24], chúng tôi cho thấy hiệu suất được cải thiện so với RTC thời gian suy luận đối với hai nhiệm vụ rất phức tạp: đóng hộp và pha cà phê espresso.

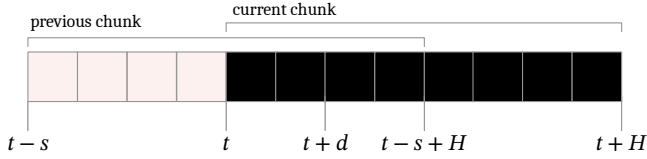
II. CÔNG VIỆC LIÊN QUAN

Phân đoạn hành động và VLA. Phân đoạn hành động [9, 26] là tiêu chuẩn thực tế trong việc học bắt chước từ đầu đến cuối để kiểm soát vận động thị giác. Gần đây, việc tăng cường các mô hình ngôn ngữ tầm nhìn (VLM) để tạo ra các khối hành động đã chứng tỏ thành công lớn trong việc thao tác robot, tạo ra các mô hình hành động-ngôn ngữ tầm nhìn (VLA) [4, 6–8, 10–12, 14, 17, 28, 29]. Sau đó, rất nhiều phương pháp đã xuất hiện để giải quyết sự căng thẳng giữa các VLA lớn và điều khiển tần số cao. Ví dụ: Gemini Robotics [23] và GR00T [3] sử dụng các thiết kế VLA phân cấp trong đó mô hình được chia thành thành phần Hệ thống 2 hạng nặng (lập kế hoạch cấp cao) và thành phần Hệ thống 1 hạng nhẹ (tạo hành động cấp thấp). MiniVLA [2] và SmolVLA [20] trình bày các kiến trúc VLA hoàn toàn nhanh hơn và hiệu quả hơn hầu hết các thiết kế, giúp cho việc suy luận ở biên trở nên khả thi hơn. Những đóng góp này trực giao với đóng góp của chúng tôi và đi kèm với sự đánh đổi của riêng chúng (ví dụ: kiến trúc mạng đã sửa đổi và công thức đào tạo).

Thực thi VLA theo thời gian thực. Công việc trước đây có liên quan chặt chẽ nhất là phân đoạn thời gian thực (RTC; [5]), giới thiệu một khung thực thi không đồng bộ làm nền tảng cho công việc này. Cũng liên quan là SmolVLA [20], trình bày thuật toán thực thi không đồng bộ tương tự như thuật toán RTC; tuy nhiên, SmolVLA không giải quyết được vấn đề gián đoạn giữa các khối, dẫn đến hiện tượng “giật” ngoài phân phối giữa các khối. Đồng thời với công việc này, A2C2 [19] và VLASH [22] đều giải quyết vấn đề gián đoạn bằng cách thêm một đầu hiệu chỉnh nhẹ và bằng cách điều chỉnh tương ứng một hành động duy nhất trong tương lai. Ngược lại với VLASH, chúng tôi dựa vào tiền tố đầy đủ của các hành động trong tương lai.

III. SƠ BỘ

Chúng tôi sử dụng cách trình bày vấn đề tương tự như RTC [5]: chúng tôi bắt đầu bằng chính sách phân chia hành động được biểu thị bằng $p(\mathbf{A}_t | \mathbf{O}_t)$, trong đó



Hình 1: Sơ đồ minh họa hai khối hành động chồng chéo. Các hành động d giữa t và $t + d$, được lấy từ đoạn trước, là tiền tố hành động (màu đỏ). Từ sơ đồ, chúng ta có thể dễ dàng thấy rằng chúng ta phải thỏa mãn ràng buộc $t + d \leq t - s + H \rightarrow d \leq H - s$ để có tiền tố hành động hợp lệ. Lưu ý rằng RTC tại thời điểm suy luận sử dụng tất cả $H - s$ hành động chồng chéo (màu đỏ và màu vàng) để hướng dẫn tạo đoạn hiện tại, trong khi RTC tại thời điểm đào tạo chỉ sử dụng hành động d đầu tiên (màu đỏ).

$\mathbf{A}_t = [\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}]$ là một đoạn hành động trong tương lai, $\mathbf{o}_t$ là quan sát và t biểu thị dấu thời gian của bộ điều khiển. Chúng tôi gọi H *chân trời dự đoán* và tại thời điểm suy luận, chúng tôi triển khai từng đoạn cho $s \leq H$ dấu thời gian, trong đó s là *chân trời thực thi*.

Để giải thích cho suy luận mô hình, chúng tôi xác định đại lượng d là độ trễ suy luận tính theo đơn vị dấu thời gian của bộ điều khiển. Nếu suy luận bắt đầu ở bước t thì đoạn hành động kết quả sẽ không có sẵn cho đến bước $t + d$ và do đó, hành động d đầu tiên thực sự không thể được thực thi. Tuy nhiên, miễn là $d \leq H - s$, những dấu thời gian d đầu tiên này sẽ tương ứng với các hành động từ đoạn trước đó có thể được thực thi trong thời gian chờ đợi. Chúng tôi gọi những hành động d này từ đoạn trước trùng với đoạn hiện tại là *tiền tố hành động* (xem Hình 1).

Chúng tôi xem xét các chính sách được đào tạo bằng cách khớp luồng có điều kiện [13], giúp giảm thiểu tổn thất sau:

$$\mathbf{A}_t^\tau = \tau \mathbf{A}_t + (1 - \tau) \epsilon \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (1)$$

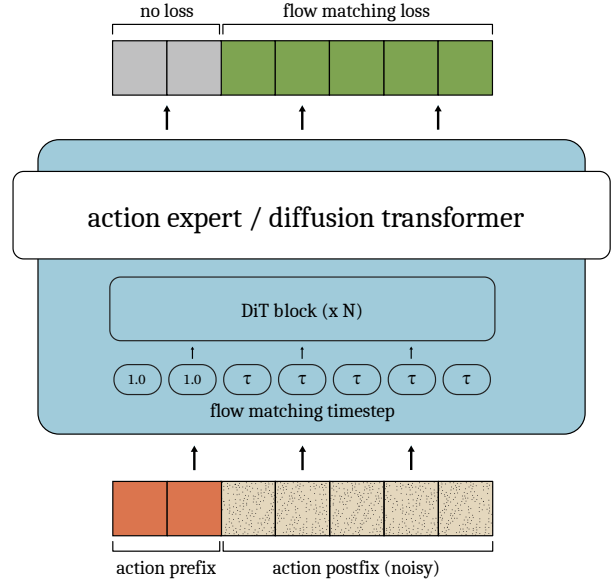
$$\mathcal{L}(\theta) = \mathbb{E} \|\mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{o}_t, \tau) - (\epsilon - \mathbf{A}_t)\|^2 \quad (2)$$

trong đó $\mathbf{v}_\theta$ là mạng nơ-ron và τ biểu thị dấu thời gian khớp luồng. Tại thời điểm suy luận, $\mathbf{v}_\theta$ có thể được tích hợp từ $\tau = 0$ đến 1 để tạo ra các mẫu từ phân phối tập dữ liệu $p(\mathbf{A}_t | \mathbf{o}_t)$.

IV. ĐIỀU KIỆN HÀNH ĐỘNG-THỜI ĐÀO TẠO

RTC thời gian suy luận [5] tạo điều kiện cho chính sách về tiền tố hành động (Hình 1, màu đỏ) bằng cách sử dụng phương pháp inpainting theo thời gian suy luận dựa trên hướng dẫn giả đảo [18, 21]. Để cải thiện tính liên tục giữa các khối, RTC thời gian suy luận bổ sung điều kiện cho *tất cả* các hành động chồng chéo, sử dụng trọng số giảm theo cấp số nhân cho các hành động ngoài tiền tố (Hình 1, màu vàng). Trong RTC, điều này được gọi là “mặt nạ mềm”. Mặc dù hướng dẫn giả nghịch đảo mang lại tính linh hoạt cao - cho phép tạo mặt nạ mềm - nhưng nó cũng yêu cầu tính toán tích vector Jacobian (sử dụng lan truyền ngược) trong mỗi bước khử nhiễu.

Cái nhìn sâu sắc cốt lõi của công việc này là chúng ta có thể điều chỉnh chính sách về tiền tố hành động tại thời điểm đào tạo bằng cách mô phỏng độ trễ suy luận. Mặc dù điều này không mang lại sự linh hoạt tương tự như việc inpainting theo thời gian suy luận, nhưng nó giúp loại bỏ chi phí tính toán. Về mặt hình thức, chúng ta có thể tìm hiểu $p(\mathbf{A}_{t+d:H} | \mathbf{o}_t, \mathbf{A}_{t:t+d})$, trong đó $\mathbf{A}_{t:t+d}$ là tiền tố hành động (Hình 1, màu đỏ) và $\mathbf{A}_{t+d:H}$ là tiền tố hành động (Hình 1, màu vàng và xanh lục), cả hai đều được lấy từ cùng một đoạn hành động thực tế. Việc triển khai điều này cho hầu hết các kiến trúc chính sách tiêu chuẩn chỉ yêu cầu 3 thay đổi tối thiểu:



Hình 2: Minh họa về kiến trúc điều hòa của chúng tôi, khi được áp dụng cho máy biến áp khuếch tán tiêu chuẩn chẳng hạn như chuyên gia hành động $\pi_{0.6}$. Chúng tôi luôn cung cấp các hành động tiền tố chân thực, không ồn ào, đồng thời học cách khử nhiễu các hành động tiền tố. Dấu thời gian khớp luồng khác nhau giữa các mã thông báo, điều này cho biết độ trễ suy luận của mô hình.

- 1) Sửa đổi kiến trúc mô hình để cho phép dấu thời gian khớp luồng khác nhau cho từng dấu thời gian hành động. Đối với kiến trúc giống như máy biến áp khuếch tán [16], sử dụng điều hòa adaLN-zero cho dấu thời gian khớp luồng, điều này không quan trọng - chỉ cần cho phép tỷ lệ, độ dịch chuyển và công khác nhau giữa các mã thông báo. Điều này không thay đổi số lượng tham số có thể học được.
- 2) Sử dụng các hành động chân thực, không ồn ào cho tiền tố và đặt dấu thời gian khớp luồng tương ứng thành 1. Không thay đổi bất kỳ điều gì cho tiền tố. Điều này tạo điều kiện cho mô hình trên tiền tố hành động thực tế trong khi chỉ sử dụng nó để khử nhiễu hậu tố.
- 3) Che giấu hàm mất mát để tổn thất chỉ được tính trên các đầu ra tương ứng với hậu tố.

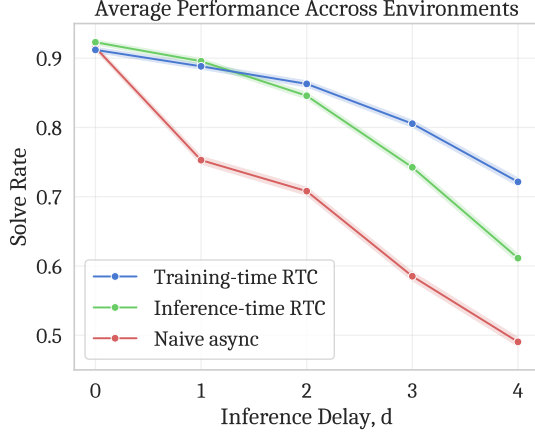
Xem Hình 2 để biết minh họa về sơ đồ điều hòa này khi được áp dụng cho kiến trúc giống như máy biến áp khuếch tán tiêu chuẩn (ví dụ: chuyên gia hành động $\pi_{0.6}$). Xem Thuật toán 1 để biết mã Python triển khai đầy đủ tính toán tổn thất và tạo hành động. Trong thực tế, vì chúng tôi không biết trước chính xác độ trễ suy luận (và độ trễ suy luận trong thế giới thực có thể khác nhau), nên chúng tôi lấy mẫu d ngẫu nhiên trong quá trình đào tạo.

Với những sửa đổi này, việc tạo hành động lấy đầu vào là tiền tố hành động $\mathbf{A}_{t:t+d}$ và chính độ trễ d và tạo ra đầu ra là tiền tố hành động $\mathbf{A}_{t+d:H}$. Do đó, nó tuân thủ cùng giao diện với thành phần tạo hành động của RTC thời gian suy luận (xem [5], Thuật toán 1) và do đó hoạt động như một sự thay thế thả vào liền mạch.

V. THÍ NGHIỆM

Trong các thử nghiệm của mình, chúng tôi mong muốn so sánh RTC thời gian đào tạo với RTC thời gian suy luận, cũng như với RTC đồng bộ và đơn giản.

đường cơ sở không đồng bộ. Các thử nghiệm mô phỏng của chúng tôi sử dụng cùng điểm chuẩn động Kinetix [15] như RTC (xem [5] để biết chi tiết). Các thử nghiệm trong thế giới thực của chúng tôi được xây dựng trên $\pi_{0.6}$ mô hình cơ sở [24] và bao gồm hai nhiệm vụ chính xác và đầy thử thách: đóng hộp và pha cà phê espresso. Chúng tôi sử dụng thiết lập thử nghiệm tương tự như $\pi_{0.6}^*$ [1].



Hình 3: Kết quả mô phỏng: độ trễ suy luận so với tốc độ giải quyết với thời gian thực thì cố định là $s = \max(d, 1)$. RTC thời gian đào tạo hoạt động tốt hơn RTC thời gian suy luận ở độ trễ suy luận từ 2 trở lên. Mỗi điểm dữ liệu đại diện cho 2048 thử nghiệm và khoảng điểm Wilson 95% được tô bóng.

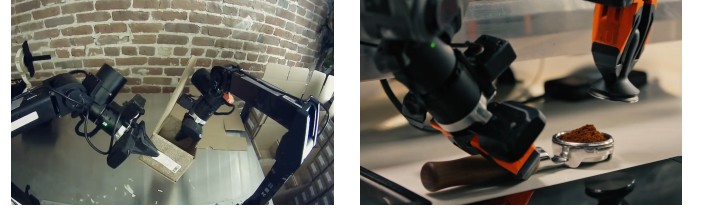
A. Kết quả mô phỏng

Trong điểm chuẩn Kinetix động, theo RTC [5], chúng tôi huấn luyện các chính sách luồng phân đoạn hành động với phạm vi dự đoán là $H = 8$ và kiến trúc MLP-Mixer 4 lớp [25] cho 32 kỳ nguyên trên dữ liệu được tạo bởi sự kết hợp của các chính sách chuyên gia. Chúng tôi báo cáo tỷ lệ thành công nhị phân với 2048 lần triển khai trên mỗi điểm dữ liệu và độ trễ thử nghiệm trong khoảng từ 0 (vòng kín hoàn toàn) đến 4 (được hỗ trợ tối đa khi $H = 8$). Cả RTC không đồng bộ và thời gian suy luận đều sử dụng cùng một điểm kiểm tra, được huấn luyện bình thường mà không điều hòa tiền tố hành động trong 32 kỳ nguyên.

Đối với RTC trong thời gian đào tạo, chúng tôi tiếp tục đào tạo từ kỳ nguyên thứ 24 và tinh chỉnh trong 8 kỳ nguyên với điều kiện tiền tố hành động. Chúng tôi làm điều này để tất cả các phương pháp đều phù hợp trong điện toán đào tạo. Chúng tôi lấy mẫu độ trễ từ $\{0, 1, 2, 3, 4\}$ với trọng số giảm dần theo cấp số nhân, vì chúng tôi nhận thấy rằng độ trễ càng cao thì cần ít sự giám sát đào tạo hơn. Có thể đạt được kết quả tốt hơn bằng cách dành nhiều thời gian đào tạo hơn cho các điểm kiểm tra đào tạo tính toán riêng lẻ cho từng độ trễ.

Các kết quả được trình bày trong Hình 3. Chúng tôi nhận thấy rằng RTC trong thời gian huấn luyện hoạt động tốt hơn RTC thời gian suy luận ở độ trễ suy luận từ 2 trở lên - với khoảng cách ngày càng mở rộng khi độ trễ tăng lên. Điều này có thể là do khi kích thước của tiền tố tăng lên, thuật toán inpainting phải “làm việc chăm chỉ hơn” để tạo ra một hậu tố nhất quán. Trong những trường hợp này, thuật toán thời gian huấn luyện mạnh hơn thuật toán thời gian suy luận thuần túy, dựa trên sự tuyến tính hóa thu được từ Jacobian của mô hình. RTC trong thời gian đào tạo hoạt động kém hơn một chút ở độ trễ 1 và 0, có thể là do RTC trong thời gian đào tạo không phải lúc nào cũng nhận được sự giám sát đào tạo cho mọi hành động - tức là

việc đào tạo tính toán ít hơn một chút để học cách tạo ra hành động đầu tiên và thứ hai.



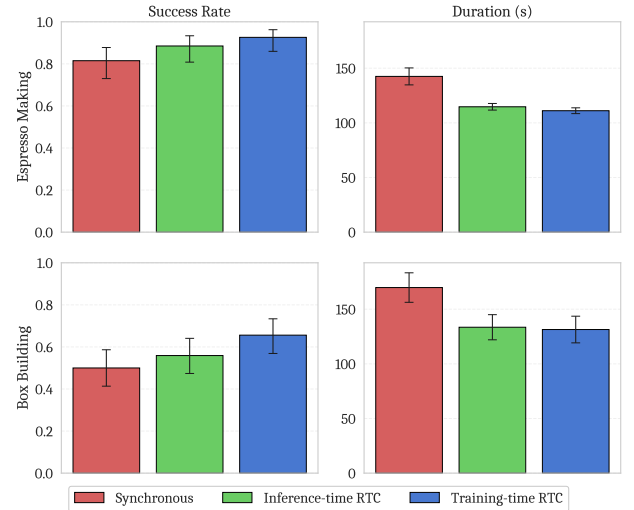
(a) Box building task

(b) Espresso making task

Hình 4: Nhiệm vụ đánh giá trong thế giới thực: đóng hộp các tông và pha cà phê espresso (bao gồm xay, nén, chiết và rót).

B. Kết quả thực tế

Trong các thử nghiệm trong thế giới thực, chúng tôi sử dụng $\pi_{0.6}$ mô hình cơ sở [24] và thử nghiệm các nhiệm vụ pha cà phê espresso và đóng hộp từ $\pi_{0.6}^*$ [1]; xem Hình 4 để minh họa. Như trong các thử nghiệm mô phỏng, chúng tôi sử dụng cùng một điểm kiểm tra cho RTC cơ sở và thời gian suy luận đồng bộ, đồng thời huấn luyện điểm kiểm tra thứ hai với điều hòa tiền tố hành động cho RTC thời gian huấn luyện. Cả hai điểm kiểm tra đều được tinh chỉnh từ mô hình cơ sở của nhiệm vụ mục tiêu cho 8.000 bước chuyển màu với kích thước lô là 512. Chúng tôi lấy mẫu độ trễ đồng đều trong khoảng từ 0 đến 10 trong quá trình đào tạo, hỗ trợ độ trễ tối đa là 200 mili giây trên rô-bốt 50Hz. Trong quá trình đánh giá, chúng tôi thực hiện suy luận trên máy chủ H100 từ xa với 5 bước khử nhiễu, độ trễ trung bình là 108ms từ đầu đến cuối cho RTC thời gian đào tạo ($d \approx 5$) và 135ms cho RTC thời gian suy luận ($d \approx 7$).



Hình 5: Kết quả trong thế giới thực: tỷ lệ thành công và thời gian pha cà phê espresso và đóng hộp. RTC thời gian đào tạo và thời gian suy luận hoạt động tương tự nhau, đồng thời cả hai đều cải thiện tốc độ so với suy luận đồng bộ. Các thanh lỗi biểu thị khoảng điểm Wilson 68% cho tỷ lệ thành công và ± 1 SEM cho thời lượng.

Các kết quả được trình bày trong Hình 5. Chúng tôi thấy rằng RTC trong thời gian đào tạo duy trì cả hiệu suất và tốc độ tương đương với RTC trong thời gian suy luận mà không cần bất kỳ chi phí tính toán nào. Cả hai biến thể của RTC đều cải thiện rõ ràng tốc độ qua mạng đồng bộ

Đường cơ sở suy luận, thể hiện các khoảng dừng có thể nhìn thấy ở giữa các đoạn.

VI. THẢO LUẬN VÀ CÔNG VIỆC TƯƠNG LAI

Trong công việc này, chúng tôi đã trình bày một giải pháp thay thế đơn giản và hiệu quả cho phân đoạn thời gian thực (RTC) giúp loại bỏ mọi chi phí tính toán trong thời gian suy luận bằng cách thêm một lượng nhỏ điện toán đào tạo bổ sung. Phương pháp của chúng tôi không yêu cầu sửa đổi kiến trúc mô hình hoặc thời gian chạy của robot và có thể được triển khai chỉ với một vài dòng mã bổ sung. Các thử nghiệm mô phỏng của chúng tôi cho thấy rằng RTC trong thời gian huấn luyện hoạt động tốt hơn RTC trong thời gian suy luận ở độ trễ suy luận cao hơn, trong khi các thử nghiệm trong thế giới thực của chúng tôi cho thấy rằng RTC trong thời gian huấn luyện duy trì cả hiệu suất và tốc độ tương đương với RTC trong thời gian suy luận mà không cần bất kỳ chi phí tính toán nào.

Tuy nhiên, RTC thời gian đào tạo về cơ bản kém linh hoạt hơn RTC thời gian suy luận; nó chỉ hỗ trợ điều hòa tiền tố hành động “cứng” tương ứng với độ trễ suy luận, trong khi RTC thời gian suy luận có thể kết hợp “nhẹ nhàng” các hành động bổ sung ngoài tiền tố. Ngoài ra, RTC trong thời gian đào tạo yêu cầu lựa chọn cẩn thận cách phân bổ độ trễ để mô phỏng tại thời điểm đào tạo dựa trên độ trễ suy luận dự kiến. Chúng tôi mong muốn công việc trong tương lai có thể giải quyết những hạn chế này và kết hợp những điều tốt nhất của cả hai thế giới.

VII. LỜI CẢM ƠN

Chúng tôi cảm ơn Laura Smith vì đã phát triển các nhiệm vụ pha cà phê espresso và giúp đánh giá sớm. Chúng tôi cảm ơn Brian Ichter vì đã phản hồi về bản thảo. Như mọi khi, chúng tôi cảm ơn toàn bộ nhóm vận hành robot vì những đóng góp của họ trong việc thu thập và đánh giá dữ liệu.

TÀI LIỆU THAM KHẢO

- [1] Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, Danny Driess, và những người khác. $\pi_{0.6}^*$: một vla học hỏi từ kinh nghiệm. *arXiv preprint arXiv:2511.14759*, 2025.
- [2] Suneel Belkhole và Dorsa Sadigh. Minivla: Một vla tốt hơn với quy mô nhỏ hơn, vào năm 2024. URL <https://github.com/Stanford-ILIAD/openvla-mini>.
- [3] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, và những người khác. Gr00t n1: Mô hình nền tảng mở cho robot hình người tổng quát. *arXiv preprint arXiv:2503.14734*, 2025.
- [4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, và những người khác. π_0 : Mô hình luồng hành động-ngôn ngữ-tầm nhìn để điều khiển robot nói chung. *bản in trước arXiv arXiv:2410.24164*, 2024.
- [5] Kevin Black, Manuel Y Galliker và Sergey Levine. Thực thi các chính sách luồng phân đoạn hành động theo thời gian thực. *arXiv bản in trước arXiv:2506.07339*, 2025.
- [6] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding,

Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alex Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Hương Trần, Vincent Vanhoucke, Quân Vương, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, và Brianna Zitkovich. Rt-2: Các mô hình hành động-ngôn ngữ-tầm nhìn chuyển kiến thức web sang điều khiển robot. Trong *bản in trước arXiv arXiv:2307.15818*, 2023.

[7] Chi-Lam Cheang, Guanzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, Hanbo Zhang và Minzhao Zhu. Gr- 2: Mô hình hành động-ngôn ngữ video tổng hợp với kiến thức ở quy mô web để thao tác với robot. *arXiv preprint arXiv:2410.06158*, 2024.

[8] An-Chieh Cheng, Yandong Ji, Zhaojing Yang, Xueyan Zou, Jan Kautz, Erdem Biyik, Hongxu Yin, Sifei Liu và Xiaolong Wang. NaVILA: Tầm nhìn Robot có chân- Mô hình hành động ngôn ngữ để điều hướng. *arXiv bản in trước arXiv:2412.04453*, 2024.

[9] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023. [10] OX-Embodiment Collaboration, A Padalkar, A Pooley, A Jain, A Bewley, A Herzog, A Irpan, A Khazatsky, A Rai, A Singh, et al. Open X-Embodiment: Robotic learning datasets and RT-X models. *arXiv preprint arXiv:2310.08864*, 1(2), 2023.

[11] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Es- mail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. [12] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[13] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022. [14] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024. [15] Michael Matthews, Michael Beukman, Chris Lu, and Jakob Foerster. Kinetix: Investigating the training of

general agents through open-ended physics-based control tasks. *arXiv preprint arXiv:2410.23208*, 2024. [16] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023.

[17] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quân Vương, Oier Mees, Chelsea Finn và Sergey Levine. Nhanh: Mã thông báo hành động hiệu quả cho các mô hình hành động-ngôn ngữ-tầm nhìn. *arXiv preprint arXiv:2501.09747*,

[18] Ashwini Pople, Matthew J Muckley, Ricky TQ Chen, and Brian Karrer. Training-free linear image inverses via flows. *arXiv preprint arXiv:2310.04432*, 2023. [19] Kohei Sendai, Maxime Alvarez, Tatsuya Matsushima, Yutaka Matsuo, and Yusuke Iwasawa. Leave no observation behind: Real-time correction for vla action chunks. *arXiv preprint arXiv:2509.23224*, 2025.

[20] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel ArActingi, Caroline Pascal, Martino Russi, Andrew Marafioti, et al. Smolvla: Mô hình hành động-ngôn ngữ-thị giác dành cho robot hiệu quả và giá cả phải chăng. *arXiv preprint arXiv:2506.01844*,

[21] Bài hát Gia Minh, Arash Vahdat, Morteza Mardani và Jan Kautz. Các mô hình khuếch tán có hướng dẫn giả nghịch đảo cho các bài toán nghịch đảo. Trong *Hội nghị quốc tế về đại diện học tập*, 2023.

[22] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. Vlash: Real-time vlas via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025.

[23] Nhóm Robot Gemini, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gonzalez Arenas, Travis Armstrong, Ashwin Balakrishna, Robert Baruch, Maria Bauza, Michiel Blokzijl, và những người khác. Gemini Robotics: Đưa con mắt vào thế giới vật chất. *arXiv Bản in trước arXiv:2503.20020*,

[24] Physical Intelligence team. $\pi_{0.6}$ model card. 2025. [25] Ilya O Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, et al. Mlp-mixer: An all-mlp architecture for vision. *Advances in neural information processing systems*, 34: 24261–24272, 2021.

[26] Tony Z Zhao, Vikash Kumar, Sergey Levine và Chelsea Finn. Học thao tác hai tay chi tiết bằng phần cứng chi phí thấp. *arXiv Preprint arXiv:2304.13705*,

[27] Tony Z Zhao, Jonathan Tompson, Danny Driess, Pete Florence, Kamyar Ghasemipour, Chelsea Finn, and Ayzaan Wahid. Aloha unleashed: A simple recipe for robot dexterity. *arXiv preprint arXiv:2410.13126*, 2024. [28] Haoyu Zhen, Xiaowen Qiu, Peihao Chen, Jincheng Yang, Xin Yan, Yilun Du, Yining Hong, and Chuang Gan. 3d-

VLA: Mô hình thể giới sáng tạo hành động-ngôn ngữ hình ảnh 3D. *arXiv Bản in trước arXiv:2403.09631*, [29] Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daume III, Andrey Kolobov, Furong Huang và Jianwei Yang. Tracevla: Nhắc nhở theo dõi trực quan nâng cao nhận thức về không gian-thời gian cho các chính sách robot tổng quát. *arXiv preprint arXiv:2412.10345*,

Algorithm 1 Mã Python triển khai các hàm mất và lấy mẫu để điều hòa hành động trong thời gian huấn luyện. Sự khác biệt từ stanMã phù hợp với dòng chảy Dard được tô sáng màu đỏ.

```
import jax
import jax.numpy as jnp

def compute_loss(rng, model, observation, action_chunk, max_delay):
    b, ah, ad = action_chunk.shape # (batch_size, action_horizon, action_dim)
    noise_rng, time_rng, delay_rng = jax.random.split(rng)
    time = jax.random.uniform(time_rng, (b,))
    noise = jax.random.normal(noise_rng, (b, ah, ad))
    # sample delays from some distribution of choice:
    # here, we use Unif[0, max_delay), as in our real-world experiments
    delay = jax.random.randint(delay_rng, (b,), 0, max_delay)

    # set time to 1.0 for the action prefix
    # time becomes shape (batch_size, action_horizon)
    prefix_mask = jnp.arange(ah)[None, :] < delay[:, None]
    time = jnp.where(prefix_mask, 1.0, time[:, None])

    # compute the noisy action postfix and run the model
    x_t = time[:, :, None] * action_chunk + (1 - time[:, :, None]) * noise
    pred_v_t = model(observation, x_t, time)
    loss = (pred_v_t - (action_chunk - noise))**2

    # compute the loss on the postfix only
    postfix_mask = jnp.logical_not(prefix_mask)[:, :, None]
    loss = jnp.sum(loss * postfix_mask) / (jnp.sum(postfix_mask) + 1e-8)
    return loss

def sample_actions(rng, model, observation, action_prefix, delay, num_steps):
    # assume action_prefix is padded to (batch_size, action_horizon, action_dim),
    # but only the first delay actions are valid
    b, ah, ad = action_prefix.shape
    x_t = jax.random.normal(rng, (b, ah, ad))
    time = 0.0
    dt = 1 / num_steps
    prefix_mask = jnp.arange(ah)[None, :] < delay

    for _ in range(num_steps):
        x_t = jnp.where(prefix_mask[:, :, None], action_prefix, x_t)
        time_masked = jnp.where(prefix_mask, 1.0, time)
        v_t = model(observation, x_t, time_masked)
        x_t = x_t + dt * v_t
        time = time + dt

    return x_t
```
